BÀI TẬP CÁ NHÂN 3

Tác vụ bất đồng bộ và tính duy nhất khi retry

Thông tin	Nội dung
Sinh viên	Phạm Nhật Quang - 23001920
Nhóm / vai trò	SE2026-T11 / Thành viên
Đề tài / biến thể	AI-02 - Document Intelligence cho biểu mẫu tiếng Việt / VT-042
Repository	github.com/teamfuturetime/document-inteligence
Baseline commit	db727496312087af6495aa12e7c9a52df3dfa230
Commit test / sửa lỗi	9a76a6ed5d06 / 6289d84e3da0
Ngày kiểm chứng	03/11/2026

Nhánh nộp bài: codex/bt3-vt042-async-retry

1. Đặc tả thay đổi

Khi OCR kéo dài hoặc phản hồi upload bị mất, người dùng cần theo dõi tác vụ đã nhận mà không tạo thêm bản ghi khi gửi lại. Baseline đã có hàng đợi và worker; thay đổi này bảo vệ retry tại thời điểm hai request tranh vị trí cuối trong hàng đợi.

ID	Tiêu chí chấp nhận
AC1	Upload hợp lệ trả 202 kèm document_id/run_id; GET đọc được queued, running và trạng thái kết thúc, độc lập với inference.
AC2	Cùng key và fingerprint, kể cả retry đồng thời hoặc mất ACK sau commit, giữ cùng IDs; đúng một document, một run và một uploaded event.
AC3	Key cũ khác fingerprint trả 409; key mới khi đầy trả 429; không thêm record/event hoặc để lại file thừa.
AC4	Lỗi OCR, lỗi ghi file và restart có trạng thái xác định; retry upload không tự tạo run mới; reload đọc lại dữ liệu lưu.
AC5	Giới hạn B byte là inclusive: B-1/B được nhận; B+1 trả 413 và không phát sinh dữ liệu.

Bất biến và ràng buộc phi chức năng

Mỗi idempotency key hợp lệ nhận diện tối đa một logical upload đã commit; capacity chỉ áp dụng với yêu cầu mới. Fingerprint gồm SHA-256 của bytes, loại tài liệu và nguồn dữ liệu.

NFR1: kiểm chứng 150 request đồng thời, so sánh p95 với 4,0 giây và báo riêng tỷ lệ nhận/từ chối. NFR2: admission atomic trên SQLite, không giữ transaction trong lúc inference. NFR3: chỉ dùng synthetic/sample, tests tách DB và điều phối đồng thời xác định.

Cam kết: “Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI.”

2. Repository và kế hoạch triển khai

Sau git fetch origin --prune, đối chiếu 10 remote branch và 412 file implementation. Bảng dưới ghi head tại thời điểm bắt đầu; origin/HEAD là alias. Audit đầy đủ lưu trong evidence/remote-audit.json.

Branch	Head	Hành vi liên quan
bt3-vt043-autosave	55e9c74daed4	Autosave OCR bằng revision/CAS; phạm vi lưu nháp, khác admission/retry.
chore/ci-checks	55ae3c6ccfb2	Bounded upload → validate ở threadpool → mock invoice; chưa có job bền vững.
congtri	5e184457fe5d	Route process gọi process_document và lưu extraction trước khi trả 202.
feat/backend-skeleton	a2810298ff71	Health/error handlers; chưa có admission hoặc worker.
hw3/23001948-vt045-config-contract	69106d4e9424	Policy hóa đơn được tách thành config/contract; không sửa queue/retry.
main	94c5101b94a4	Chỉ README; chưa có luồng xử lý tài liệu.
minhthao	997da9fe1cf6	Có auth/ownership/review; process await HTTP AI trong request.
nhatquang	db7274963120	SQLite queued/running/terminal, worker riêng, UNIQUE key; UI polling 1.200 ms.
personal/canhtung-main	61a31ab5888b	OCR thật và model pin; /extract vẫn await run_document trong threadpool.
tuannguyen	52067697c1e4	Pipeline/OCR/benchmark CLI; HTTP API tối thiểu, chưa durable job.

Kế hoạch trước sửa code

Prompt: “Trên baseline db727496, sinh test VT-042 cho async upload và retry; giữ SQLite/file thật, dùng Barrier ép initial key miss, kiểm IDs/counts/files, kiểm biên B-1/B/B+1 và lưu red trước patch.”

Thứ tự: test/spec → commit test → sửa quyết định admission → green/mutation/coverage/tải → hồi quy và native/frontend → commit, push nhánh riêng. File sửa: backend/tests/test_async_retry_vt042.py và backend/src/docintel/services/documents.py; runner, README và logs ở homework/VT-042. API/schema, model/data/config giữ nguyên.

Điểm trong plan được review	Quyết định từ source
Thêm Celery/Redis và bảng jobs	Bác bỏ: runs/status và LocalInferenceWorker đã có. Chỉ re-read key trong write transaction hiện hữu.
async def/202 nghĩa là background	Bác bỏ: congtri/minhthao/canhtung còn chờ xử lý trong request; nhatquang đã tách persisted admission và worker.
Status-only đủ cho retry; mutate queue >=	Sửa oracle thành IDs/counts/files; mutation > thành >= đặt tại upload byte limit, vì queue >= hiện đúng.

Điều kiện triển khai: một SQLite DB/worker, PNG synthetic, cùng key khi retry. Rủi ro ngoài phạm vi: thông báo/thanh toán chưa có trong baseline, nên không kết luận về tính duy nhất của side effect bên ngoài.

3. Chiến lược kiểm thử theo rủi ro

Rủi ro	Tác động / ưu tiên	Lớp kiểm thử
Retry sau commit hoặc request đồng thời	Record/run/event trùng hoặc replay sai; P0	API + SQLite/file thật; forced race và mất ACK
Key trùng nhưng fingerprint khác	Ghi đè dữ liệu hoặc trả sai 429; P0	API: payload, loại tài liệu và nguồn dữ liệu
OCR/file lỗi, worker gián đoạn	Kẹt trạng thái hoặc tạo tác vụ mới khi retry; P0	Service/DB: fault injection, restart/recovery
Biên kích thước / trạng thái UI	Off-by-one, không biết tiến trình; P0/P1	API B-1/B/B+1; E2E upload → polling → reload

Unit kiểm thuật toán chọn tổ hợp. Integration/API kiểm transaction, constraint, file và error contract. Một E2E đi qua browser, OCR thật và persistence; tải TCP được đo riêng. Không kiểm pixel-perfect UI, payment hoặc notification cho slice này; OCR accuracy benchmark không phải oracle của retry.

Pairwise / combinatorial testing

Chọn bốn factor nhị phân: loại tài liệu (hóa đơn/đơn), key (cũ/mới), payload (giống/khác seed) và queue (còn chỗ/đầy). Có $2^4 = 16$ tổ hợp khả thi. Thuật toán greedy chọn hàng phủ nhiều cặp chưa có nhất; khi hòa lấy hàng đầu theo thứ tự product cố định.

Case	Loại	Key	Payload	Queue	Expected
PW01	Hóa đơn	Cũ	Giống	Còn	202, replay
PW02	Hóa đơn	Mới	Khác	Đầy	429
PW03	Đơn	Cũ	Giống	Đầy	202, replay
PW04	Đơn	Mới	Khác	Còn	202, tạo mới
PW05	Hóa đơn	Cũ	Khác	Còn	409
PW06	Hóa đơn	Mới	Giống	Còn	202, tạo mới

Sáu hàng phủ 24/24 cặp giá trị: $C(4,2) \times 2^2$. test_pairwise_covers_every_value_pair so sánh tập cặp phủ với toàn bộ 16 tổ hợp, không chỉ đếm số case. Các tương tác ba/bốn chiều không được suy ra từ pairwise; hai race tại capacity được bổ sung có chủ đích.

Dữ liệu và oracle

PNG trắng/đen 12×12 được sinh cố định để phân biệt hash; không chứa dữ liệu cá nhân. Admission tạo (documents,runs,events,revisions) = (1,1,1,0), hai file source/preview. Retry phải giữ IDs, nội dung cũ và cardinality. Provider/decode được cô lập trong suite; SQLite, transaction và file I/O vẫn thật. Native OCR có phép kiểm riêng.

4. Các test cases và kết quả mong đợi

24 node pytest và một hành trình browser. (-): negative test; (F): failure path. P0 bảo vệ tính duy nhất, dữ liệu và trạng thái; P1 bổ sung khả năng sử dụng và tính tương thích. File: backend/tests/test_async_retry_vt042.py; mapping tên hàm/dòng: evidence/test-map.json.

ID	AC	Level	Precondition / input	Expected result	Ưu tiên
PW01-06	AC2/3	API/DB	Tổ hợp ở trang 3, seed 1 run; cap 1/2.	Status, IDs replay và DB/file counts đúng.	P0
T07	AC2	Unit	Sinh covering array từ 16 hàng.	Đủ 24/24 cặp, chọn 6 hàng.	P1
T08/W1	AC2	API/DB	Gửi cùng K và bytes hai lần.	Cùng document/run; (1,1,1,0), 2 files.	P0
T09/W2	AC3	API/DB	K cũ, PNG khác bytes.	409; hash/run/files cũ nguyên vẹn.	P0
T10	AC2	API/DB	Barrier 2 request cùng K; cap=1.	Hai 202; một document/run/event.	P0
T11	AC3	API/DB	Cùng race, hai payload khác.	Một 202, một 409; không có 429.	P0
T12	AC5	API	Body B-1 byte.	202; đúng một upload.	P1
T13	AC5	API	Body đúng B byte.	202; bắt mutant >=.	P0
T14 (-)	AC5	API	Body B+1 byte.	413; không record/event/file.	P0
T15 (-)	AC3	API	Key có space và dấu !.	422; không side effect.	P1
T16 (-)	AC3	API/DB	Queue đầy; key mới.	429 QUEUE_FULL; counts không tăng.	P0
T17 (F)	AC4	Worker/DB	Pipeline ném OCR_TIMEOUT; retry K.	failed rõ; giữ run, không enqueue lại.	P0
T18	AC1	API/worker	Event chặn inference; GET ở giữa.	Đọc running; thả Event → terminal.	P0
T19 (F)	AC4	File/DB	Source đã ghi; preview ném OSError.	Xóa source, DB rỗng; retry thành công.	P0
T20	AC4	DB/restart	Một queued, một running; service mới.	Queued giữ; running bị đánh dấu interrupted.	P0
T21 (F)	AC2	API/DB	Response hook làm mất ACK sau commit.	Retry cùng IDs; vẫn (1,1,1,0), 2 files.	P0
T22/23 (-)	AC3	API/DB	K cũ, khác loại tài liệu hoặc origin.	409; run/dữ liệu/file không đổi.	P0
T24 (F)	AC4	API/DB	Provider unavailable.	503; không admission/file.	P0
E25	AC1/4	Browser	Tải hoa_don_007.png bằng OCR thật.	UI Đã xử lý; reload còn một document/run.	P1

T21 mô phỏng mất ACK tại client response hook sau khi endpoint commit; đây không phải phép gây lỗi proxy/WAN. T18 dùng Event để kiểm running trước khi inference trả về, không dựa vào thời gian OCR cụ thể. Mỗi test có tmp_path riêng.

5. Test-first implementation và review diff

Mốc Git	Thay đổi / kết quả
db7274963120	Baseline từ origin/nhatquang; đã có async worker và key UNIQUE.
9a76a6ed5d06	Commit test/spec trước patch. Red: 2 failed, 22 passed; cả hai forced race trả 429 sai contract.
6289d84e3da0	Thêm 19 dòng ở service upload. Green: 24 passed (2,62 s); hồi quy backend/AI: 96 passed (16,72 s).

Commit test

Commit sửa lỗi

Nguyên nhân lỗi

Initial SELECT nằm ngoài write transaction. Hai request có thể cùng chưa thấy key; request đầu commit run chiếm slot cuối. Request sau bị check_capacity trả 429 trước khi tới UNIQUE INSERT và nhánh xử lý IntegrityError. Baseline không tạo record trùng trong hai case này, nhưng trả sai kết quả retry/conflict.

Patch tại services/documents.py::upload

```
with self.repository.transaction(write=True) as connection:
    replay = (
        connection.execute(
            "SELECT * FROM documents WHERE idempotency_key=?", (idempotency_key,)
        ).fetchone()
        if idempotency_key
        else None
    )
    if replay:
        if (replay["input_sha256"], replay["document_type"], replay["data_origin"]) != (
            digest,
            document_type,
            data_origin,
        ):
            raise ProductError(
                409, "IDEMPOTENCY_CONFLICT", "Khóa đã dùng cho tài liệu khác."
            )
        self.storage.discard(document_id)
        return self.detail(replay["document_id"])
    self.check_capacity(connection)
```

Tra lại key dưới BEGIN IMMEDIATE trước capacity. Nếu đã có: đối chiếu fingerprint, loại file source/preview của request thua rồi trả lại bản đã commit. Nếu key mới: giữ nguyên capacity, INSERT và UNIQUE. Full diff: evidence/fix.patch.

Review và phạm vi

Scope: một method service; không đổi contract/dependency. Design: DB tiếp tục quyết định admission. Correctness: kiểm hash/type/origin trước replay, giữ cleanup và trạng thái lỗi. Verification: hai race, mất ACK, fingerprint mismatch, file fault và suite hồi quy. Nhánh riêng chứa test/patch/README, không merge vào main.

6. Đánh giá chất lượng bộ test

Mutation: > thành >=

Tạm đổi len(data) + len(chunk) > maximum thành >= maximum ở api/documents.py. Ba test B-1/B/B+1 chạy độc lập: B bị 413 thay vì 202, hai case lân cận vẫn pass. Kết quả: 1 failed, 2 passed, 21 deselected (0,91 s). T13 phát hiện mutation; finally khôi phục bytes API, sau đó 24 tests pass (2,98 s). Chỉ một mutant được thử, không suy ra mutation score toàn dự án.

Coverage của 24 tests VT-042

Module	Dòng thực thi	Nhánh
api/documents.py	42/52 = 80.8%	4/4 = 100.0%
repositories/sqlite.py	36/37 = 97.3%	1/2 = 50.0%
services/documents.py	121/239 = 50.6%	17/58 = 29.3%

coverage.py 7.17.0, branch=True. API đạt đủ bốn nhánh được công cụ ghi nhận nhưng còn endpoint chưa chạy; service còn nhiều review/export/insights ngoài slice. Bộ 96 tests hồi quy chạy riêng, không cộng vào coverage này. Assertion, mutation và mapping rủi ro được dùng cùng tỷ lệ coverage.

Hai test AI yếu được sửa

Draft	Thay đổi trong suite
W1 chỉ assert upload trả 202	T08 gửi lại cùng key, so document/run IDs, counts DB/event/revision và hai file; phát hiện được duplicate dù status vẫn đúng.
W2 lặp happy path bằng key khác	T09 đổi thành conflicting payload: 409 và hash/run/files của tài liệu cũ không bị thay đổi.

Kiểm tra độ ổn định

Nguy cơ	Cách kiểm soát / kết quả
Thời gian và concurrency	Barrier ép cả hai initial miss; Event giữ inference để GET running. Không sleep để đoán tiến độ; timeout chỉ giới hạn test treo.
Thứ tự và shared state	tmp_path/DB riêng, fixture khôi phục monkeypatch. Ba lượt thường và một lượt đảo: đều 24 passed, thời gian 2,45-2,47 s.
Network và random data	Suite API in-process, không WAN; PNG/pairwise cố định, UUID kiểm theo quan hệ. TCP load tách thành harness riêng.

7. Kiểm chứng tải và luồng người dùng

Harness performance.py phát 150 coroutine HTTP cùng một Event qua TCP loopback tới Uvicorn; giới hạn connection là 150, peak in-flight ghi nhận đúng 150. Mỗi scenario dùng DB/temp directory mới. Lượt đo cuối chạy tuần tự sau các checks; lượt đo khi checks còn hoạt động cũng được lưu riêng.

Scenario	Peak	HTTP / số lượng	p50 / p95 / p99 (s)	DB D/R/E
150 key mới	150	202: 5; 429: 145	0.448 / 1.215 / 1.465	5/5/5
150 retry cùng key	150	202: 150	0.424 / 0.473 / 0.479	1/1/1
150 GET trạng thái	150	200: 150	0.340 / 0.398 / 0.404	1/1/1

Phương pháp đo

Windows 11, CPython 3.14.5, CPU Intel64 Family 6 Model 141; httpx2 2.13.1, Uvicorn 0.54.0. Đo từ trước client.post/get đến khi nhận response. p95 dùng nearest-rank ceil(0,95×150), tức mẫu thứ 143; gồm mọi status và lỗi, không loại 429. Không có retry ẩn.

PNG synthetic 80 byte; queue capacity mặc định 5, worker tắt. Readiness/decode là doubles công khai; TCP, endpoint, SQLite transaction và file I/O thật. Retry/GET có một seed upload trước lượt đo. Toàn bộ request/latency lưu ở evidence/performance.json.

Đối chiếu mục tiêu 4,0 giây

Cả ba p95 phản hồi API quan sát được dưới 4,0 giây. Với 150 key mới, chỉ 5 request được nhận, 145 trả 429 do queue đầy; DB đúng 5/5/5. Với 150 retry, tất cả trả 202 và vẫn chỉ 1/1/1. GET trạng thái không tạo thêm record/event.

Kết quả chứng minh phản hồi admission/retry/status trong điều kiện thử, không chứng minh 150 tác vụ OCR hoàn tất trong 4 giây. Mẫu nhỏ, một máy local và một lượt mỗi scenario giới hạn khả năng suy rộng; queue hiện chỉ cho max_pending_runs ≤ 20 và một worker/DB.

E2E với OCR thật

Browser tải hoa_don_007.png, nhận kết quả qua polling và hiển thị “Đã xử lý”; sau reload còn đúng một document/run. Native workflow tự động đạt 14/14 checks. Một ảnh là smoke của luồng xử lý, không phải benchmark OCR. DOM/ảnh và metadata được lưu trong evidence/browser-*.

8. Truy vết và tự đánh giá

AC / Risk	Test case	Test / evidence	Kết quả
AC1 / tiến trình	T18, E25; tải GET	Test + browser-e2e.json	GET running; native UI succeeded/reload pass
AC2 / trùng dữ liệu	PW01/03, T08/10/21	Test; performance.py	24 suite pass; retry150 giữ DB 1/1/1
AC3 / conflict/capacity	PW02/04/05/06; T09/11/15/16/22/23	Test; red/green.log	409/429/422, counts và dữ liệu cũ đúng
AC4 / failure/restart	T17/19/20/24, E25	Test; native-workflow.json	Fault/recovery pass; native 14/14
AC5 / off-by-one	T12/13/14	Test; mutation.json	Biên pass; T13 giết mutant >=

Test = backend/tests/test_async_retry_vt042.py. Commit test: 9a76a6ed5d06; commit patch: 6289d84e3da0. README và toàn bộ evidence đi kèm trên nhánh nộp bài; mapping tên hàm/dòng ở test-map.json.

Tự đánh giá

Điều làm tôi tin tưởng hơn ở thay đổi này là khả năng tái hiện lỗi trước khi sửa. Hai request cùng khóa đều vượt qua lần đọc ban đầu, nhưng request đến sau bị từ chối vì hàng đợi đầy. Barrier tạo đúng điều kiện đó mà không phụ thuộc vào sleep. Khi quyết định retry được đặt trong transaction ghi, cả hai request nhận cùng định danh, cơ sở dữ liệu vẫn chỉ có một tài liệu, một run và một event.

Một điểm đáng chú ý là UNIQUE trong database chưa đủ bảo vệ toàn bộ contract. Constraint giữ được tính duy nhất nhưng thứ tự kiểm tra capacity vẫn có thể tạo phản hồi sai. Test chỉ assert status thành công sẽ không nhìn thấy số record hoặc file thừa. Vì vậy, tôi ưu tiên oracle dựa trên định danh, fingerprint và dữ liệu đã commit. Mutation tại giới hạn upload cung cấp một phép kiểm khác: case đúng bằng maximum phát hiện toán tử bị đổi dù hai giá trị lân cận vẫn pass.

Kết quả tải cũng cần được đọc cùng tỷ lệ thành công. Phản hồi nhanh bằng 429 không tương đương xử lý đủ 150 tác vụ; phép đo hiện tại cô lập OCR và chỉ xác nhận admission, retry, đọc trạng thái. Một mẫu OCR thật trên browser bổ sung bằng chứng tích hợp nhưng không thay thế phép đo ảnh đa dạng hoặc môi trường nhiều máy. Tôi đánh giá cao việc lưu cả red, patch, green và logs theo commit vì có thể lần lại từng kết luận. Test tiếp theo có giá trị nhất là mất phản hồi qua proxy trong lúc worker OCR thật chạy, đồng thời restart tiến trình, rồi kiểm tra tính duy nhất của kết quả và lịch sử xử lý.

Tài liệu tham khảo: [1] Bùi Sỹ Nguyên, Lecture 06 - AI-assisted Software Development & Vibe Coding, slide 9-18, 21-23. [2] Lecture 07 - Software Testing & Quality Engineering, slide 7-12, 16, 18-24. [3] Repository AI-02, các commits và branch được liệt kê ở trang 1-2.

PHỤ LỤC A - AI Usage Log

Công cụ: Codex; Git CLI, uv/pytest/ruff/coverage, Uvicorn/httpx2, browser CUA và ReportLab. AI được dùng cho kế hoạch, test cases, patch nhỏ, thí nghiệm và biên soạn báo cáo; quyết định chấp nhận dựa trên source/diff và kết quả thực thi.

Tác vụ / prompt chính	Output sử dụng	Review / thay đổi / evidence
Fetch/prune và phân tích tất cả remote branch; tạo nhánh VT-042 riêng rồi push.	Audit 10 branches; chọn baseline db727496, branch codex/bt3-vt042-async-retry.	Phân biệt admission/retry với autosave VT-043 và config-contract VT-045; remote-audit.json.
Sinh test async/retry; giữ SQLite/file thật, Barrier ép miss, kiểm IDs/counts/files; biên B-1/B/B+1; red trước patch.	24 tests; pairwise 6 hàng/24 cặp, failure paths, ACK loss.	W1 status-only → IDs/cardinality; W2 happy path lặp → conflict bất biến. Test/spec commit 9a76a6ed5d06.
Review plan bằng source và chọn patch nhỏ.	19 dòng re-read key trong transaction trước capacity.	Bác thêm queue framework/schema; bác async/202 = background; kiểm fingerprint/cleanup và giữ UNIQUE. Patch commit 6289d84e3da0.
Mutate > thành >=; chạy lại test; đo coverage và lặp/đảo tests.	Mutation killed, API bytes restored; coverage.json; 4 lượt suite pass.	Không mutate queue >= đang đúng. Runner thử đầu thiếu prefix backend nên exit 4; sửa tham số và chạy lại, giữ log thử đầu.
Đo tải TCP150 và kiểm luồng native/frontend/browser.	Raw request latencies, status/counters; native 14 checks, UI 7 tests, browser reload.	Giữ 429 trong mẫu; tách OCR khỏi API load; lưu lượt tải khi checks đang chạy và lượt chạy tuần tự.
Cập nhật báo cáo theo commits và logs trên branch.	8 trang nội dung, 3 phụ lục; Times New Roman, đen trắng.	Thay thông tin nhánh thiếu bằng URL thực; bỏ hướng dẫn bàn giao, giữ AI Usage Log và giới hạn phép đo theo đề.

Hai draft test đã thay

```
# W1 - draft status-only
def test_retry_status_only(api):
    assert upload(api).status_code == 202

# W2 - draft lặp happy path
def test_upload_again(api):
    assert upload(api, key="OTHER").status_code == 202
```

Draft ban đầu được lưu từ lượt 30/10; suite ngày 03/11 giữ oracle đã sửa và bổ sung bốn case. Hai điểm cần kiểm chứng thay vì tin output AI: async/202 không tự chứng minh background, và một tài liệu lưu hai file source/preview. File spec-and-plan.md và AI_USAGE_LOG.md giữ prompt, assumptions, file dự kiến và các quyết định.

PHỤ LỤC B - Logs thực thi

B1. Baseline red → patch green

```
uv run --project backend --locked pytest backend/tests/test_async_retry_vt042.py -q
test_concurrent_retry_at_full_capacity_replays_the_winner:
    actual [202, 429] != expected [202, 202]
test_concurrent_conflicting_key_at_capacity_returns_conflict:
    actual [202, 429] != expected [202, 409]
2 failed, 22 passed in 6.79s          # red.log

Sau patch: 24 passed in 2.62s         # green.log
Hồi quy:   96 passed in 16.72s        # regression.log
```

B2. Mutation và khôi phục

```
uv run --project backend --locked python homework/VT-042/experiments.py mutation
test_upload_byte_boundary_is_inclusive[0-202]
    assert 413 == 202
1 failed, 2 passed, 21 deselected in 0.91s
# mutation-killed.log, exit 1
API bytes restored; 24 passed in 2.98s
# mutation-restored.log, exit 0
```

B3. Coverage và thứ tự test

```
uv run --project backend --locked --with coverage==7.17.0 python
    homework/VT-042/experiments.py coverage
uv run --project backend --locked python homework/VT-042/experiments.py repeat
repeat-1: 24 passed in 2.47s
repeat-2: 24 passed in 2.45s
repeat-3: 24 passed in 2.45s
reverse:  24 passed in 2.46s
```

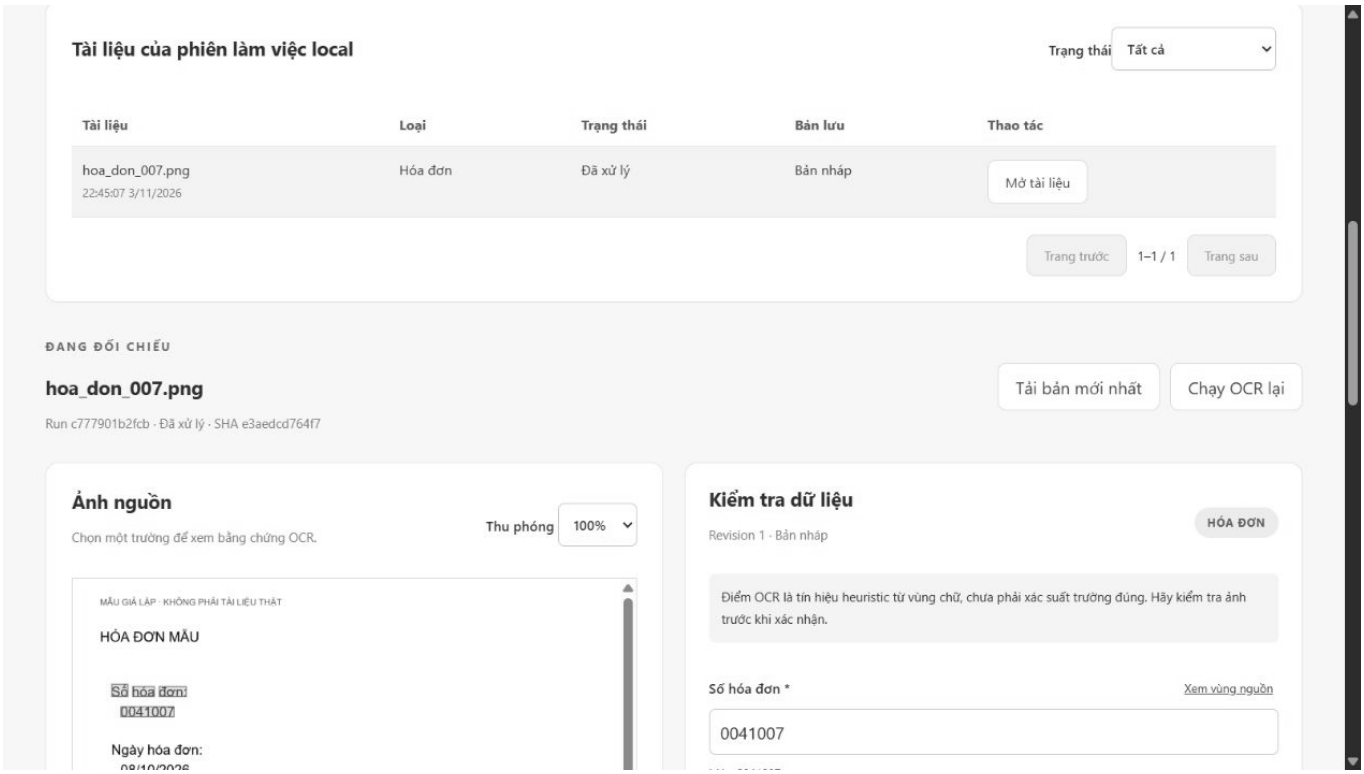
B4. Kiểm tra bổ sung

Lệnh / nhóm checks	Kết quả
ruff check; ruff format --check backend/scripts/pipeline	Pass; 60 files tại lượt kiểm chính
generate_contracts.py --check; frontend contracts:check	25 contracts + frontend types không drift
check_data.py; evaluate_fixtures.py; check_repository.py	6 fixtures + 150 parent samples; evaluator contract/docs pass
verify_product.py	14 checks passed, native Tesseract thật
npm contracts/typecheck/lint/format/test/build	Pass; 7 tests frontend
npm audit (dependency tree hiện hữu)	15 advisories: 13 moderate, 2 high; lockfile giữ nguyên

Lệnh đầy đủ và exit code: checks-backend.json, checks-frontend.json; tracebacks và output nguyên dạng: các file .log. Workflow của baseline chạy trên PR hoặc push main; push nhánh bài tập không kích hoạt workflow. Kết quả trên là checks local, không phải remote CI.

PHỤ LỤC C - E2E native trên browser

03/11/2026: build frontend từ nhánh bài làm, tải ảnh synthetic hoa_don_007.png; Tesseract xử lý và UI nhận kết quả qua polling. Sau reload: một document, một run succeeded; hai events là uploaded và processed. Run ID: c777901b2fcb4d388f524c5b1c6d1593.



OCR engine: 5.4.0.20240606; model revision 87416418657359cb625c412a48b6e1d6d41c29bd. Metadata, hashes frontend/model/source và DOM trước/sau reload nằm trong browser-e2e.json và browser-*.txt. Ảnh này là minh chứng workflow của một mẫu; không dùng để suy ra độ chính xác OCR hoặc hiệu năng 150 tác vụ.

README chạy test và thí nghiệm

Test suite VT-042